ملاحظات المراجعة — لك

الخوارزميات وهندسة البرمجيات: اختبار الوحدة

البرمجة، المستوى ٤، اختبار الوحدة

المدة
٢٠ دقيقة

المواد
ورق وقلم. لا حاجة إلى جهاز.

أهداف التعلّم

- كيف ينمو العمل: O الكبيرة
- الاستدعاء الذاتي
- هياكل البيانات
- الاختبار بعقلية المهندس

النشاط: حلقة مراجعة الكود

١. التمهيد (٣ دقائق): اكتب بكلماتك فكرة واحدة من كل درس من الدروس الأربعة.
٢. الكتابة (٧ دقائق): اكتب دالة صغيرة من هذه الوحدة (first_repeat أو balanced أو digit_sum أو search) مع خمسة أسطر assert، ومنها حالات حدّية.
٣. المراجعة (٧ دقائق): مرّر كودك إلى الشخص التالي، فيضيف اختبارًا يظنّ أنه سيكسره، وتعليقًا مفيدًا واحدًا.
٤. الإصلاح (٣ دقائق): استعد كودك واجعل كل الاختبارات تنجح، ومنها الاختبار الجديد.

للنقاش

- أيّ فكرة من هذه الوحدة ستستعملها في مشروعك القادم؟
- أيّ سؤال فاجأك، ولماذا؟
- أين تهمّ O الكبيرة في تطبيق حقيقي تستعمله؟

تحقّق من نفسك

- عادة واحدة سأحافظ عليها مبرمجًا

جرّبها هذا الأسبوع

خذ برنامجًا كتبته هذا العام. حدّد O الكبيرة له، واستبدل جزءًا بطيئًا منه بهيكل البيانات المناسب، واحمِه بخمسة أسطر assert، منها حالات حدّية.

ورقة الاختبار

الخوارزميات وهندسة البرمجيات: اختبار الوحدة

البرمجة، المستوى ٤، اختبار الوحدة

الاسم _____ الفصل _____ التاريخ _____

١ خوارزمية من $O(n^2)$ تستغرق ثانية واحدة لـ ١٠٠٠ عنصر. كم تستغرق تقريبًا لـ ٢٠٠٠ عنصر؟

◯ نحو ثانية واحدة ◯ نحو ٤ ثوانٍ ◯ نحو ثانيتين

٢ البحث الثنائي في قائمة مرتّبة من O(log n).

◯ صح ◯ خطأ

٣ اكتب has_pair(values, target): تعيد True إن كان مجموع عنصرين مختلفين يساوي target. ويجب أن تبقى سريعة مع ٢٠٠٠٠ قيمة، فاستعمل مجموعة: لكل v، هل رأيت target − v من قبل؟

```
def has_pair(values, target):
    # one pass with a set of the values seen so far
    return False
```

اكتب برنامجك:

٤ يجب أن تكون fact(4) هي 24، لكن هذه تعيد 0. أيّ سطر فيه الخطأ؟

ضع دائرة حول السطر الذي فيه الخطأ، ثم اكتبه صحيحًا:

```
1  def fact(n):
2      if n == 1:
3          return 0
4      return n * fact(n - 1)
```

٥ صِل كل عملية بكلفتها أو بما تعطيه.

x in a_list	خطوة تقريبًا
x in a_set	يفحص واحدًا واحدًا: O(n)
stack.pop()	أقدم عنصر
queue.pop(0)	أحدث عنصر

٦ دون تذكّر النتائج، كم مرة تحسب fib(5) القيمة fib(2)؟

○ 1 ○ 3 ○ 5

٧ رتّب الأسطر: عُدّ حروف «banana» بقاموس واطبع عدد حرف a، أي 3. وفيها سطر لا مكان له.

رقّم الأسطر بالترتيب (١، ٢، ٣...)، وارسم أمام السطر سهمًا ← عن كل إزاحة يحتاج إليها. اشطب أي سطر لا ينتمي إلى البرنامج.

☐ `for c in "banana":`

☐ `count = {}`

☐ `print(count["a"])`

☐ `count[c] = 1`

☐ `count[c] = count.get(c, 0) + 1`

٨ أيّها صحيح عن العمل بالاختبار أولًا؟

☐ تستطيع تخطّي الحالات الحدّية إن بدا الكود صحيحًا ☐ لا تنظّف الكود إلّا والاختبارات كلها تنجح

☐ يجب أن يفشل الاختبار الجديد في البداية ☐ تكتب الاختبار قبل الكود

٩ املأ الفراغات في فيبوناتشي مع تذكّر النتائج.

```
memo = {}
def fib(n):
    if n < 2:
        return n
    if n not in _____:
        memo[n] = fib(n - 1) + fib(n - 2)
    return memo[_____]
```

بنك الكلمات: `fib` `memo` `n - 1` `n`

١٠ رتّب خطوات البحث الثنائي.

☐ وإلّا فاحتفظ بالنصف الذي يمكن أن يحوي الهدف

☐ انظر إلى العنصر الأوسط

☐ كرّر حتى يفرغ النطاق

☐ إن كان هو الهدف فتوقّف

١١ على الهاتف، تعمل الدالة التي تستدعي نفسها دون حالة أساسية إلى الأبد.

○ صح ○ خطأ

تحقّق من إجاباتك

الخوارزميات وهندسة البرمجيات: اختبار الوحدة

البرمجة، المستوى ٤، اختبار الوحدة

١ **نحو ٤ ثوانٍ**

ضاعف n يُضرب عمل n^2 في أربعة: نحو ٤ ثوانٍ.

٢ **صح**

كل نظرة تنصّف ما بقي، فتنمو النظرات مثل log n: نحو ٢٠ لمليون عنصر.

٣

```
def has_pair(values, target):
    seen = set()
    for v in values:
        if target - v in seen:
            return True
        seen.add(v)
    return False
```

مرور واحد، O(n): لكل قيمة، الشريك الذي تحتاجه هو target − v، وتقول المجموعة في خطوة تقريبًا هل رأيته.

٤ **السطر 3: `return 1`**

تُبلغ الحالة الأساسية، لكن بجواب خاطئ: فيُضرب كل شيء في 0.

٥

O(n): `x in a_list` — يفحص واحدًا واحدًا

`x in a_set` — خطوة تقريبًا

`stack.pop()` — أحدث عنصر

`queue.pop(0)` — أقدم عنصر

القوائم تبحث واحدًا واحدًا، والمجموعات تقفز مباشرة، والمكدّس يعطي الأحدث، والطابور الأقدم.

٦ **3**

تستدعي fib(5) كلًّا من fib(4) وfib(3)، وتظهر fib(2) تحتهما، ثلاث مرات في المجموع.

٧ **من الأعلى إلى الأسفل: ٢، ١، ٤، ✗، ٣**

```
count = {}
for c in "banana":
    count[c] = count.get(c, 0) + 1
print(count["a"])
```

تعيد get(c, 0) العدد حتى الآن، فيضيف كل حرف واحدًا.

٨ **تكتب الاختبار قبل الكود، يجب أن يفشل الاختبار الجديد في البداية، لا تنظّف الكود إلّا والاختبارات كلها تنجح**

أحمر، أخضر، تحسين. والكود الذي «يبدو صحيحًا» هو بالضبط حيث تختبئ أخطاء الحالات الحدّية.

٩

```
memo = {}
def fib(n):
    if n < 2:
        return n
    if n not in memo:
        memo[n] = fib(n - 1) + fib(n - 2)
    return memo[n]
```

احسب fib(n) فقط إن لم تكن في memo، ثم أعِد الجواب المخزّن لـ n.

١٠ **من الأعلى إلى الأسفل: ٣، ١، ٤، ٢**

تدور الحلقة ما دام في النطاق lo…hi شيء: while lo <= hi.

١١ **خطأ**

يمتلئ مكدّس الاستدعاءات (عند نحو ١٠٥٠ استدعاءً في MicroPython) ويتوقّف البرنامج برسالة خطأ.